TechCorp Simulator and Parallel Foundations Track

Next Main Lesson and Parallel Beginner Lesson

Contents

1 Part I: Next Main Lesson in the Game Track **3**

1.1 Why This Lesson Matters . . . 3

1.2 Conceptual Structure of a Turn . . . 4

1.3 The GameEngine Class . . . 4

1.4 Didactic Commentary on `GameEngine` . . . 6

1.5 The ConsoleUI Class . . . 7

1.6 Why This Class Is Valuable . . . 8

1.7 Main Class Integration . . . 8

1.8 Lesson Topics in Order . . . 9

1.9 Exercises for the Main Track Lesson . . . 10

2 Part II: Parallel Beginner Lesson **11**

2.1 The Economic Problem . . . 11

3 Stage 1: Solving the Problem in `main()` **12**

3.1 Why Start in `main()` . . . 12

3.2 What This Version Teaches . . . 12

3.3 The Limit of This Approach . . . 13

4 Stage 2: Identifying What the Program Really Contains **13**

5 Stage 3: Introducing a Company Class **13**

5.1 What Has Improved . . . 15

6 Stage 4: Updating `main()` **15**

6.1 Didactic Commentary . . . 16

7 Stage 5: Optional Further Decomposition **16**
7.1 Why This Extra Step Matters 17

8 Lesson Flow for the Beginner Track **17**

9 Learning Outcomes for the Beginner Lesson **17**

10 Exercises for the Beginner Lesson **18**

11 Final Comparison Between the Two Tracks **18**
11.1 Main Game Lesson 18
11.2 Parallel Beginner Lesson 19
11.3 Why They Fit Together 19

12 Conclusion **19**

1. Part I: Next Main Lesson in the Game Track

Lesson Title

Lesson 5: Game Loop, Player Decisions, and Menu-Based Flow

Lesson Aim

The aim of this lesson is to move the project from a static demonstration of objects toward a genuine **interactive decision-making game**. Up to this point, students have learned how to model employees, projects, inheritance, polymorphism, enums, and interfaces. The natural next step is to give the player a structured way to make choices and to allow the game state to change across a sequence of turns.

This lesson introduces the idea of a **game loop**. A game loop is the repeating structure that drives the whole program. It asks for input, performs actions, updates the game state, and shows the result. In other words, this is the lesson in which the program begins to feel like an actual game rather than a collection of classes.

From a didactic point of view, this lesson is significant because it integrates several previously isolated ideas:

- conditions,
- loops,
- objects,
- methods,
- validation,
- user interaction.

1.1. Why This Lesson Matters

A turn-based decision game must have some mechanism that answers the following questions:

- What can the player do now?
- How does the player choose an action?
- What changes in the world after the action?
- When does one turn end and the next begin?
- When does the whole game end?

If those questions are not answered, there may be classes and methods, but there is not yet a game. The game loop provides the missing architecture.

1.2. Conceptual Structure of a Turn

A single turn in the game may be understood as a sequence of phases. At an introductory level, the following structure is sufficient:

1. show the current state,
2. present the available actions,
3. read the player's choice,
4. perform the chosen action,
5. advance the game by one turn,
6. check whether the game should continue.

This is simple enough for beginners to follow, yet already strong enough to support a real decision-making system.

1.3. The GameEngine Class

A useful way to organise this logic is to introduce a class called `GameEngine`. This class is not a domain object like `Employee` or `Project`. Rather, it coordinates the overall flow of the program.

```java
package com.example.techcorp.engine;

import com.example.techcorp.domain.Company;
import com.example.techcorp.domain.Project;
import com.example.techcorp.domain.ProjectStatus;
import com.example.techcorp.ui.ConsoleUI;

public class GameEngine {

    private Company company;
    private ConsoleUI ui;
    private boolean running;
    private int turn;

    public GameEngine(Company company, ConsoleUI ui) {
        this.company = company;
        this.ui = ui;
```

```java
        this.running = true;
        this.turn = 1;
    }

    public void start() {
        while (running) {
            ui.showTurnHeader(turn);
            ui.showCompanyStatus(company);
            ui.showMainMenu();

            int choice = ui.readMenuChoice();

            handleChoice(choice);

            if (running) {
                advanceTurn();
                turn++;
            }
        }
    }

    private void handleChoice(int choice) {
        switch (choice) {
            case 1 -> ui.showCompanyStatus(company);
            case 2 -> startPlannedProjects();
            case 3 -> workOnProjects();
            case 4 -> running = false;
            default -> ui.showMessage("Invalid menu option.");
        }
    }

    private void startPlannedProjects() {
        for (Project project : company.getProjects()) {
            if (project.getStatus() == ProjectStatus.PLANNED) {
                project.start();
            }
        }
        ui.showMessage("All planned projects have been started.")
            ;
    }

```

```java
    private void workOnProjects() {
        for (Project project : company.getProjects()) {
            project.workOneTurn();
        }
        ui.showMessage("Projects worked for one turn.");
    }

    private void advanceTurn() {
        if (allProjectsFinished()) {
            ui.showMessage("All projects are finished. Game over.
                ");
            running = false;
        }
    }

    private boolean allProjectsFinished() {
        if (company.getProjects().isEmpty()) {
            return false;
        }

        for (Project project : company.getProjects()) {
            if (!project.isFinished()) {
                return false;
            }
        }

        return true;
    }
}
```

1.4. Didactic Commentary on `GameEngine`

This class is important because it introduces the idea that not every class in a program represents a "thing" in the world of the game. Some classes represent **processes of control**. That distinction is central to software architecture.

The `GameEngine` class:

- stores the current turn number,
- keeps the game running while appropriate,
- asks the user for decisions,

- delegates specific work to other objects.

It does *not* itself compute employee productivity or project logic. That remains the responsibility of domain classes. This separation is good design.

1.5. The ConsoleUI Class

Because user interaction should not clutter the domain model, a separate class may be used for console input and output.

```java
package com.example.techcorp.ui;

import com.example.techcorp.domain.Company;
import com.example.techcorp.domain.Project;
import java.util.Scanner;

public class ConsoleUI {

    private final Scanner scanner = new Scanner(System.in);

    public void showTurnHeader(int turn) {
        System.out.println();
        System.out.println("=== TURN " + turn + " ===");
    }

    public void showMainMenu() {
        System.out.println("Choose an action:");
        System.out.println("1. Show company status");
        System.out.println("2. Start planned projects");
        System.out.println("3. Work on projects");
        System.out.println("4. Exit game");
    }

    public int readMenuChoice() {
        System.out.print("Enter choice: ");

        if (!scanner.hasNextInt()) {
            scanner.nextLine();
            return -1;
        }

        int choice = scanner.nextInt();
        scanner.nextLine();
```

```java
        return choice;
    }

    public void showCompanyStatus(Company company) {
        System.out.println("Company: " + company.getName());
        System.out.println("Cash: " + company.getCash());
        System.out.println("Projects:");

        for (Project project : company.getProjects()) {
            System.out.println("- " + project.getName()
                    + " | status: " + project.getStatus()
                    + " | progress: " + project.getProgress()
                    + "/" + project.getRequiredWork());
        }
    }

    public void showMessage(String message) {
        System.out.println(message);
    }
}
```

1.6. Why This Class Is Valuable

This class teaches a very important architectural lesson:

> User interaction should be separated from business logic whenever reasonably possible.

The student learns that printing, reading input, and displaying menus belong to one kind of responsibility, whereas employee behaviour, project progress, and company state belong to another.

1.7. Main Class Integration

The `Main` class becomes much simpler once the control flow is delegated to dedicated classes.

```java
package com.example.techcorp;

import com.example.techcorp.domain.Company;
import com.example.techcorp.domain.Developer;
import com.example.techcorp.domain.Manager;
import com.example.techcorp.domain.Project;
```

```java
import com.example.techcorp.domain.Tester;
import com.example.techcorp.engine.GameEngine;
import com.example.techcorp.ui.ConsoleUI;

public class Main {

    public static void main(String[] args) {
        Company company = new Company("TechCorp", 50_000);

        company.hire(new Developer("Anna", 9, 8_000));
        company.hire(new Tester("Piotr", 6, 6_500));
        company.hire(new Manager("Ewa", 7, 9_000));

        Project mobileApp = new Project("Mobile App", 30);
        Project website = new Project("Website", 20);

        for (var employee : company.getEmployees()) {
            mobileApp.addEmployee(employee);
        }

        company.startProject(mobileApp);
        company.startProject(website);

        ConsoleUI ui = new ConsoleUI();
        GameEngine engine = new GameEngine(company, ui);

        engine.start();
    }
}
```

1.8. Lesson Topics in Order

A coherent teaching order for this lesson is the following:

1. explain what a game loop is,
2. introduce the idea of turn-based flow,
3. show menu-based input,
4. introduce the `GameEngine` class,
5. introduce the `ConsoleUI` class,

6. connect everything in `Main`,
7. discuss validation of menu choices,
8. discuss end conditions of the game.

1.9. Exercises for the Main Track Lesson

Exercise 1 — Add a New Menu Option

Add a menu option that displays only projects that are not finished.

Didactic aim: to reinforce conditional logic and iteration.

Exercise 2 — Pause a Project

Add an option allowing the player to place a project on hold if the status system includes `ON_HOLD`.

Didactic aim: to connect menus with enum-based state transitions.

Exercise 3 — Count Turns

Display the turn number clearly and stop the game after a fixed number of turns, for example ten.

Didactic aim: to reinforce loop control and end conditions.

Exercise 4 — Validate Input Properly

Improve `readMenuChoice()` so that invalid input is handled more gracefully.

Didactic aim: to connect user interaction with validation and robustness.

Exercise 5 — Win Condition

Add a win condition and a loss condition. For example:

- win if all projects are finished,
- lose if cash falls below zero.

Didactic aim: to integrate domain state with control flow.

2. Part II: Parallel Beginner Lesson

Lesson Title

Beginner Parallel Lesson 2: A Simple Economic Task — First in `main()`, Then as Classes

Lesson Aim

This lesson is designed for complete beginners and develops basic programming skill in parallel to the game track. The aim is to show, in a very concrete way, how one may move from:

1. a simple procedural solution written directly in `main()`,
2. to a more structured object-oriented solution based on classes.

The lesson uses a **simple economic task**: calculating whether a small company can pay monthly costs and whether it remains profitable after expenses.

This is didactically strong for several reasons:

- it uses realistic data,
- it requires arithmetic and logical reasoning,
- it is easy to understand procedurally,
- it then becomes an excellent bridge to classes and objects.

2.1. The Economic Problem

The task is as follows:

A small company has:

- a monthly revenue,
- salary costs,
- rent,
- tool costs.

We wish to calculate:

- total monthly cost,
- monthly profit or loss,
- whether the company can operate safely.

At first, this will be solved entirely inside the `main()` function. Afterwards, the same logic will be reorganised into classes.

3. Stage 1: Solving the Problem in `main()`

3.1. Why Start in `main()`

For a complete beginner, it is often pedagogically sound to solve a problem first in the most direct way possible. This prevents premature abstraction. The student should first understand the logic of the task itself before being asked to distribute that logic across multiple classes.

The following version is therefore entirely legitimate as a first step.

```java
public class Main {

    public static void main(String[] args) {

        String companyName = "BlueSoft";
        double revenue = 40000;
        double salaries = 22000;
        double rent = 5000;
        double tools = 3000;

        double totalCost = salaries + rent + tools;
        double profit = revenue - totalCost;

        boolean profitable = profit > 0;
        boolean canOperate = revenue >= totalCost;

        System.out.println("Company: " + companyName);
        System.out.println("Revenue: " + revenue);
        System.out.println("Total cost: " + totalCost);
        System.out.println("Profit: " + profit);
        System.out.println("Profitable: " + profitable);
        System.out.println("Can operate: " + canOperate);
    }
}
```

3.2. What This Version Teaches

This version teaches several fundamental things:

- variables,
- data types,
- arithmetic operators,
- boolean expressions,
- readable output.

At this point, everything is visible in one place. That is good for a beginner because the cognitive load remains low.

3.3. The Limit of This Approach

However, once the program becomes even slightly more complicated, this style begins to break down. Questions naturally arise:

- What if we want to represent more than one company?
- What if we want to reuse the cost calculation?
- What if we want clearer separation between data and operations?

These questions motivate the transition to classes.

4. Stage 2: Identifying What the Program Really Contains

Before writing classes, the student should be asked to identify the **concepts** already present in the simple solution.

In the previous `main()` solution, we really had:

- a company,
- financial data,
- operations on that financial data.

This means that the code already contains the seeds of a class structure. The next step is therefore not arbitrary. It is a way of expressing more clearly what was already there in an implicit form.

5. Stage 3: Introducing a Company Class

A natural first class is `Company`.

```
public class Company {

```

```java
    private String name;
    private double revenue;
    private double salaries;
    private double rent;
    private double tools;

    public Company(String name, double revenue, double salaries,
                   double rent, double tools) {
        this.name = name;
        this.revenue = revenue;
        this.salaries = salaries;
        this.rent = rent;
        this.tools = tools;
    }

    public double calculateTotalCost() {
        return salaries + rent + tools;
    }

    public double calculateProfit() {
        return revenue - calculateTotalCost();
    }

    public boolean isProfitable() {
        return calculateProfit() > 0;
    }

    public boolean canOperate() {
        return revenue >= calculateTotalCost();
    }

    public String getName() {
        return name;
    }

    public double getRevenue() {
        return revenue;
    }
}
```

5.1. What Has Improved

The structure is now much better for several reasons.

First, the data belongs to the company object rather than floating freely in `main()`.

Secondly, the operations that logically belong to the company are now methods of the company:

- `calculateTotalCost()`,
- `calculateProfit()`,
- `isProfitable()`,
- `canOperate()`.

This is the first real encounter with the object-oriented principle that an object combines **state** and **behaviour**.

6. Stage 4: Updating `main()`

Once the `Company` class exists, the `main()` method becomes much cleaner.

```java
public class Main {

    public static void main(String[] args) {

        Company company = new Company(
                "BlueSoft",
                40000,
                22000,
                5000,
                3000
        );

        System.out.println("Company: " + company.getName());
        System.out.println("Revenue: " + company.getRevenue());
        System.out.println("Total cost: " + company.calculateTotalCost());
        System.out.println("Profit: " + company.calculateProfit());
        System.out.println("Profitable: " + company.isProfitable());
        System.out.println("Can operate: " + company.canOperate());
```

```
    }
}
```

6.1. Didactic Commentary

This moment is crucial. The student should see that object-oriented programming is not "more complicated code for no reason". Rather, it is a way of making the program more organised once the problem becomes conceptually richer.

The transition from the first version to the second version is therefore not merely technical. It is intellectual. It means:

> The program no longer consists only of raw calculations. It now contains a model of the thing being reasoned about.

7. Stage 5: Optional Further Decomposition

If the class version is comfortable, the lesson may take one more step and introduce a second class, for example `FinancialReport` or `CostCalculator`. This is optional and should only be done if the students are ready.

For example:

```java
public class FinancialReport {

    public static void printReport(Company company) {
        System.out.println("=== FINANCIAL REPORT ===");
        System.out.println("Company: " + company.getName());
        System.out.println("Revenue: " + company.getRevenue());
        System.out.println("Total cost: " + company.calculateTotalCost());
        System.out.println("Profit: " + company.calculateProfit());
        System.out.println("Profitable: " + company.isProfitable());
        System.out.println("Can operate: " + company.canOperate());
    }
}
```

Then `main()` becomes:

```java
public class Main {

```

```java
    public static void main(String[] args) {
        Company company = new Company("BlueSoft", 40000, 22000,
            5000, 3000);
        FinancialReport.printReport(company);
    }
}
```

7.1. Why This Extra Step Matters

This optional step introduces another fundamental idea:

> Not every class represents a business entity. Some classes organise operations or presentation.

That insight later becomes important for larger architecture, including the game project itself.

8. Lesson Flow for the Beginner Track

A coherent teaching order for this beginner lesson is:

1. present the economic task,
2. solve it directly in `main()`,
3. explain the variables and calculations,
4. identify the concepts hidden inside the procedural version,
5. introduce the `Company` class,
6. move the calculations into methods,
7. simplify `main()`,
8. optionally introduce a reporting class.

9. Learning Outcomes for the Beginner Lesson

By the end of this lesson, the student should understand:

1. how a simple real-world task may be solved procedurally,
2. why procedural code inside `main()` eventually becomes unwieldy,
3. how to recognise when data and behaviour belong together,
4. how classes improve organisation,

5. how object-oriented code may emerge naturally from a simpler first version.

10. Exercises for the Beginner Lesson

Exercise 1 — Add Another Cost

Extend the program by adding one more cost category, for example marketing.

First do it in the `main()` version, then do it in the class-based version.

Didactic aim: to compare how extension works in procedural and object-oriented forms.

Exercise 2 — Check for Loss

Add a boolean value that explicitly indicates whether the company is making a loss.

Didactic aim: to reinforce comparison operators and method design.

Exercise 3 — Create Two Companies

Create two company objects and compare their profit.

Didactic aim: to show why classes are valuable when more than one object exists.

Exercise 4 — Add Validation

In the `Company` constructor, validate that revenue and costs are not negative. Throw `IllegalArgumentException` if validation fails.

Didactic aim: to connect beginner OOP with validation and robustness.

Exercise 5 — Write a Report Method

Add a method `printSummary()` to the `Company` class that displays the key financial information.

Didactic aim: to reinforce the connection between object state and behaviour.

11. Final Comparison Between the Two Tracks

The main game-track lesson and the parallel beginner lesson serve different but complementary purposes.

11.1. Main Game Lesson

The main lesson develops:

- game architecture,
- control flow,
- turn-based logic,
- menu interaction,
- integration of previously learned OOP concepts.

11.2. Parallel Beginner Lesson

The beginner lesson develops:

- confidence with simple realistic tasks,
- the transition from procedural to object-oriented thinking,
- understanding of why classes are introduced at all,
- a gradual increase in abstraction.

11.3. Why They Fit Together

The beginner lesson prepares the student's mind for the larger game lesson. In the simpler economic task, the student sees that code may first be written in a direct way and then reorganised into classes. That same intellectual move is exactly what the game project requires at a larger scale.

12. Conclusion

These two lessons form a productive pair.

The next main lesson in the game track turns the system into a playable program by introducing the game loop, menu choices, and turn progression.

The parallel beginner lesson strengthens foundational understanding by taking a simple economic problem and showing, step by step, how a first procedural solution may evolve into an object-oriented design.

Together, they help students develop not only syntax, but judgement: the ability to recognise when a problem should remain simple and when it is time to introduce clearer structure.